

Lab 1: Introduction to LabVIEW

Objective: The main objective of this lab is to get familiar with LabVIEW basics.

1. LabVIEW Environment

This section explains how to navigate NI LabVIEW and use the block diagram, front panel, and Functions and Controls palettes.

This section includes:

- Virtual Instruments (VIs)
- Front Panel
- Controls Palette
- Controls and Indicators
- Block Diagram
- Block Diagram Terminals
- Block Diagram Nodes
- Functions Palette
- Searching for Controls, VIs, and Functions

Virtual Instruments (VIs)

LabVIEW programs are called virtual instruments, or VIs, because their appearance and operation imitate physical instruments, such as oscilloscopes and multimeters. When you create a new VI, you see two windows: the **front panel** window and the **block diagram**.

Front Panel

When you open a new or existing VI, the front panel window of the VI appears. The front panel window is the user interface for the VI. Figure 1 shows an example of a front panel window.

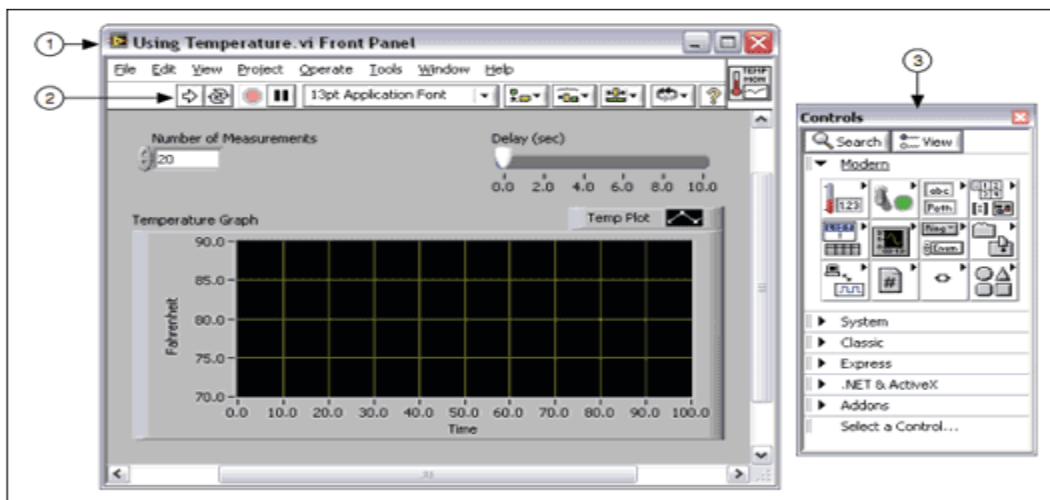


Figure 1. Front Panel Window, Toolbar, and Controls Palette

Controls Palette

The Controls palette contains the controls and indicators you use to create the front panel. You access the Controls palette from the front panel window by selecting **View»Controls Palette** or by right-clicking on any empty space in the front panel window. The Controls palette is broken into various categories;

you can expose some or all of these categories to suit your needs. Figure 2 shows a Controls palette with all of the categories exposed and the Modern category expanded.

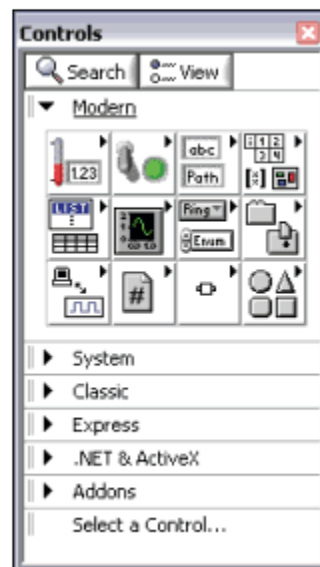


Figure 2. Controls Palette

Controls and Indicators

You create the front panel with controls and indicators, which are the interactive input and output terminals of the VI, respectively. Controls are knobs, push buttons, dials, and other input devices. Indicators are graphs, LEDs, and other displays. Controls simulate instrument input devices and supply data to the block diagram of the VI. Indicators simulate instrument output devices and display data that the block diagram acquires or generates.

Figure 1 has two controls: Number of Measurements and Delay (sec). It has one indicator: an XY graph named Temperature Graph.

The most commonly used data types are **numeric**, **Boolean value**, and **string**.

Numeric Controls and Indicators

The numeric data type can represent numbers of various types, such as integer or real. The two common numeric objects are the numeric control and the numeric indicator, as shown in Figure 3. Objects such as meters and dials also represent numeric data.

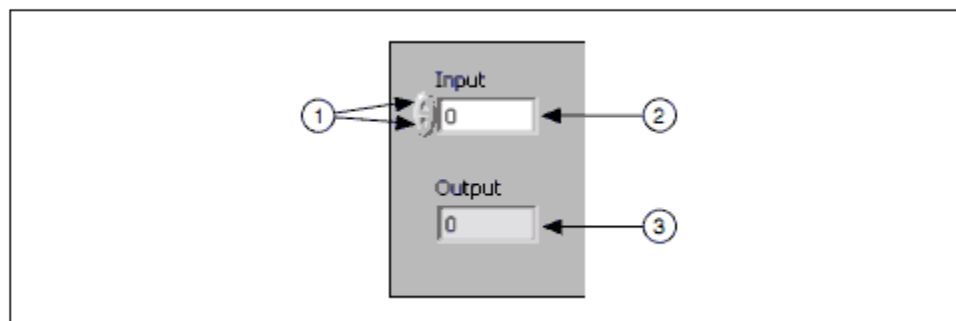


Figure 3. Numeric Control Increment/Decrement, Control, Indicator

Boolean Controls and Indicators

The Boolean data type represents data that has only two possible states, such as TRUE and FALSE or ON and OFF. Use Boolean controls and indicators to enter and display Boolean values. Boolean objects simulate switches, push buttons, and LEDs. The vertical toggle switch and the round LED Boolean objects are shown in Figure 4.

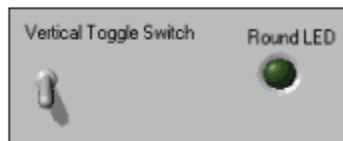


Figure 4. Boolean Controls and Indicators

String Controls and Indicators

The string data type is a sequence of ASCII characters. Use string controls to receive text from the user such as a password or user name. Use string indicators to display text to the user. The most common string objects are tables and text entry boxes, as shown in Figure 5.

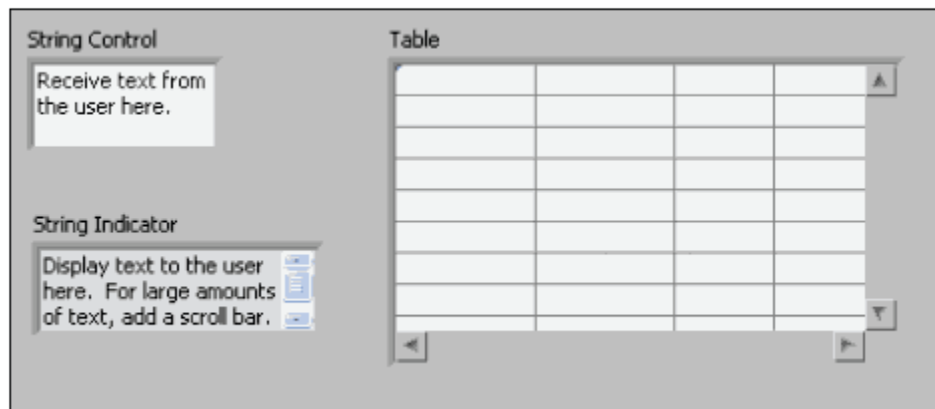


Figure 5. String Controls and Indicators

Block Diagram

Block diagram objects include terminals, subVIs, functions, constants, structures, and wires, which transfer data among other block diagram objects.

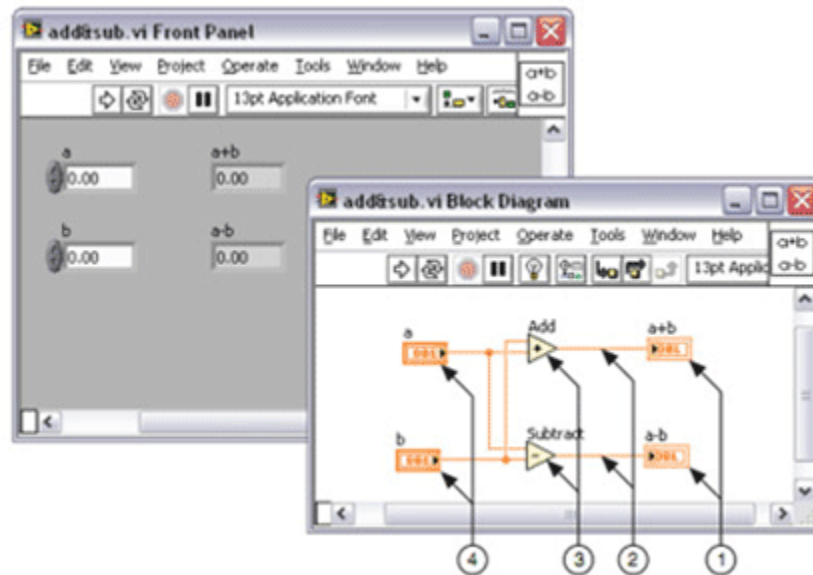


Figure 6. Indicator Terminals, Wires, Nodes, Control Terminals

Block Diagram Window

After you create the front panel window, you add code using graphical representations of functions to control the front panel objects. The block diagram window contains this graphical source code.

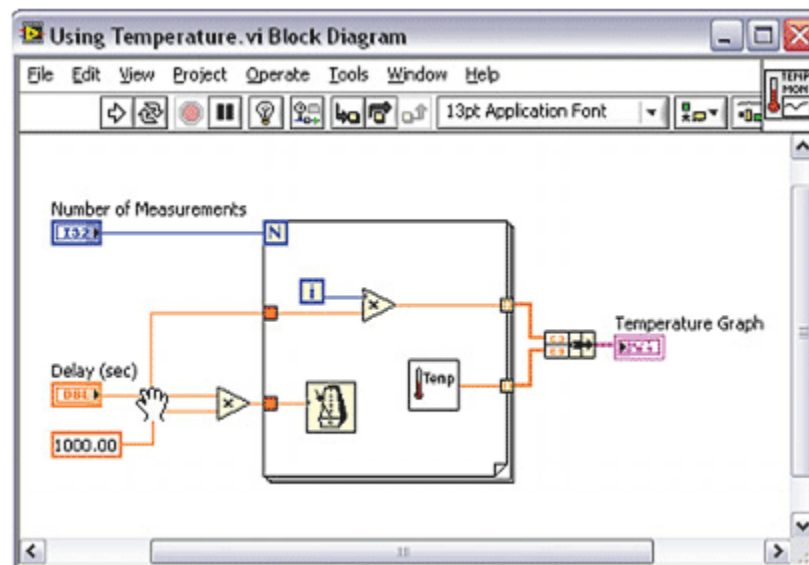


Figure 7. Block Diagram

Block Diagram Terminals

Objects on the front panel window appear as terminals on the block diagram. Terminals are entry and exit ports that exchange information between the front panel and block diagram. Data you enter into the front panel controls (a and b in the previous front panel) enters the block diagram through the control terminals. The data then enters the Add and Subtract functions. When the Add and Subtract functions complete their calculations, they produce new data values. The data values flow to the

indicator terminals, where they update the front panel indicators (a+b and a-b in the previous front panel).

Controls, Indicators, and Constants

Controls, indicators, and constants behave as inputs and outputs of the block diagram algorithm. Consider the implementation of the algorithm for the area of a triangle: $\text{Area} = .5 * \text{Base} * \text{Height}$

In this algorithm, Base and Height are inputs and Area is an output, as shown in **Figure 8**. The user does not change or access the constant .5, so it does not appear on the front panel unless included as documentation of the algorithm.

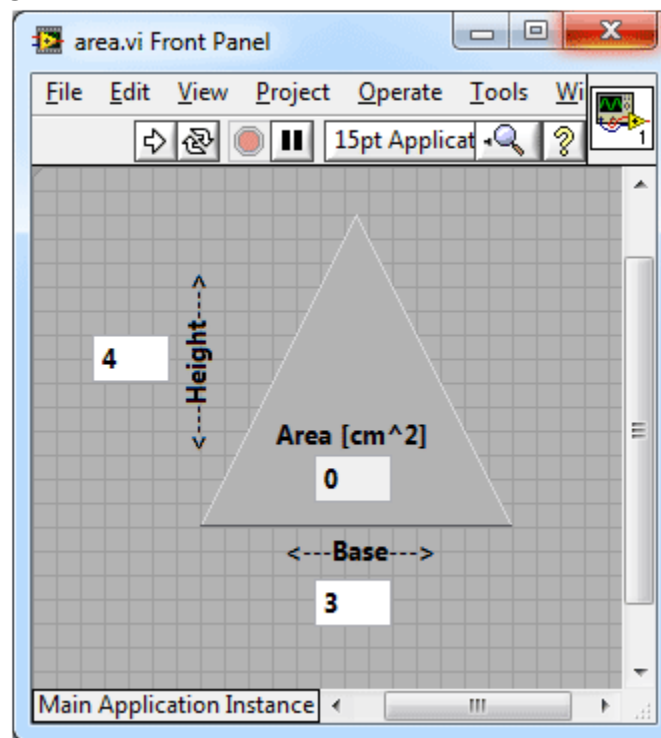


Figure 8. Area of a Triangle Front Panel

Figure 9 shows a possible implementation of this algorithm on a LabVIEW block diagram. This block diagram has four different terminals created by two controls, one constant, and one indicator.

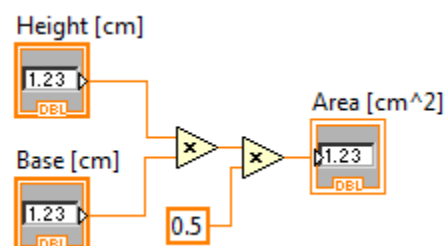


Figure 9. Control, Indicator, Constant

Block Diagram Nodes

Nodes are objects on the block diagram that have inputs and/or outputs and perform operations when a VI runs. Nodes can be **functions, subVIs, or structures**. Structures are process control elements, such as Case structures, For Loops, or While Loops. The Add and Subtract functions in Figure 6 are function nodes.

Functions

Functions are the fundamental operating elements of LabVIEW. They do not have front panel windows or block diagram windows but they do have connector panes. Double-clicking a function only selects the function. A function has a pale yellow background on its icon.

SubVIs

SubVIs are VIs that you create to use inside another VI or that you access on the Functions palette. A subVI is similar to a function in a text-based programming language. Any VI has the potential to be used as a subVI. When you double-click a subVI on the block diagram, its front panel window appears. The front panel includes controls and indicators.



Every VI displays an icon in the upper right corner of the front panel window and block diagram window. An example of the default icon is shown above. An icon is a graphical representation of a VI. The icon can contain both text and images.



To use a VI as a subVI, you need to build a connector pane, shown above. The connector pane is a set of terminals on the icon that corresponds to the controls and indicators of that VI.

SubVIs also can be **Express VIs**. Express VIs are nodes that require minimal wiring because you configure them with dialog boxes. Use Express VIs for common measurement tasks. You can save the configuration of an Express VI as a subVI.

Functions Palette

The Functions palette contains the VIs, functions, and constants you use to create the block diagram. You access the Functions palette from the block diagram by selecting **View»Functions Palette**. The palette is broken into various categories; you can show and hide categories to suit your needs. Figure 11 shows a Functions palette with all of the categories exposed and the Programming category expanded.

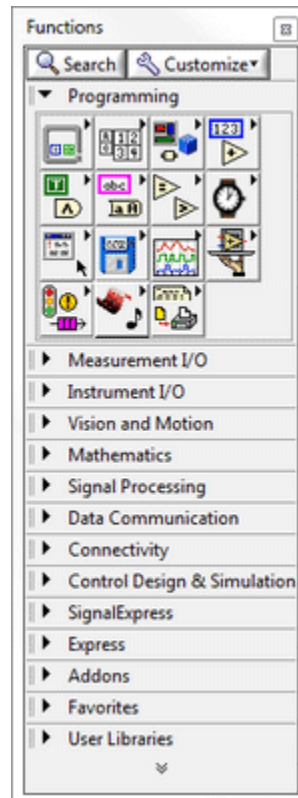


Figure 11. Functions Palette

2. Loops and Execution Structures

Execution structures contain sections of graphical code that control how and when the code inside runs. The most common execution structures are While Loops, For Loops, and Case structures. With these structures, you can run sections of code multiple times or execute a different section of code based on a condition. This section explains the functionality of these structures and when they should be used in NI LabVIEW software.

- While Loops
- Structure Tunnels
- Adding Timing to Loops
- Other Structures
- Case Structures
- Passing Data between Loop Iterations in LabVIEW

While Loops

Similar to a Do Loop or a Repeat-Until Loop in text-based programming languages, a While Loop, shown in Figure 1, executes the code it contains until a condition is met.

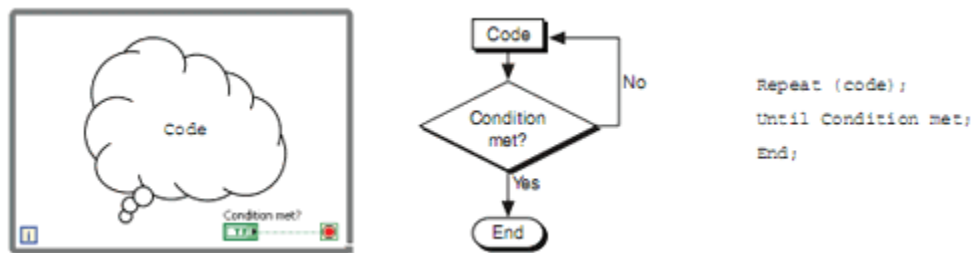


Figure 1. A While Loop in LabVIEW, a Flowchart of the While Loop Functionality, and a Pseudo Code Example of the Functionality of the While Loop

The While Loop executes the code it contains until the **Conditional Terminal**, an input terminal, receives a specific Boolean value.



The **Iteration Terminal** is an output terminal that contains the number of completed iterations. The iteration count for the While Loop always starts at zero.

Note: The While Loop always executes at least once.

Infinite Loops

Infinite loops are a common programming mistake that involves a loop that never stops. If the **Conditional Terminal** is Stop if True, you place the terminal of a Boolean control outside a While Loop, and the control is FALSE when the loop starts, you cause an infinite loop.

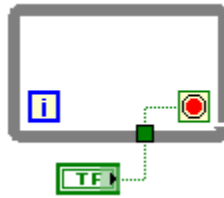


Figure 2. Boolean Control Outside While Loop

Changing the value of the control does not stop the infinite loop because the value is only read once, before the loop starts. To use a control to stop a While Loop, you should place the control terminal inside the loop. To stop an infinite loop, you must abort the VI by clicking the **Abort Execution** button on the toolbar.

Another example of infinite loop is shown below in figure 3.

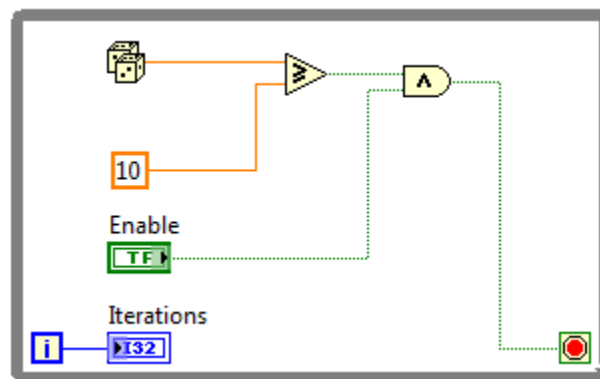


Figure 3. Infinite Loop

Structure Tunnels

Tunnels feed data into and out of structures. The tunnel appears as a solid block on the border of the While Loop. The block is the color of the data type wired to the tunnel. Data passes out of a loop after the loop terminates. When a tunnel passes data into a loop, the loop executes only after data arrives at the tunnel.

In Figure 4, the **Iteration Terminal** is connected to a tunnel. The value in the tunnel does not get passed to the Iterations indicator until the While Loop finishes executing. Only the last value of the **Iteration Terminal** displays in the Iterations indicator.

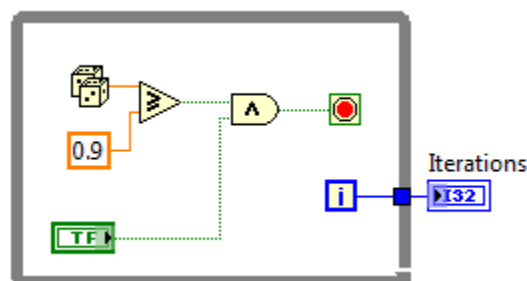


Figure 4. While Loop Tunnel

For Loops

A For Loop executes a subdiagram a set number of times. Figure 5 shows a For Loop in LabVIEW, a flowchart equivalent of the For Loop functionality, and a pseudo code example of the functionality of the For Loop.

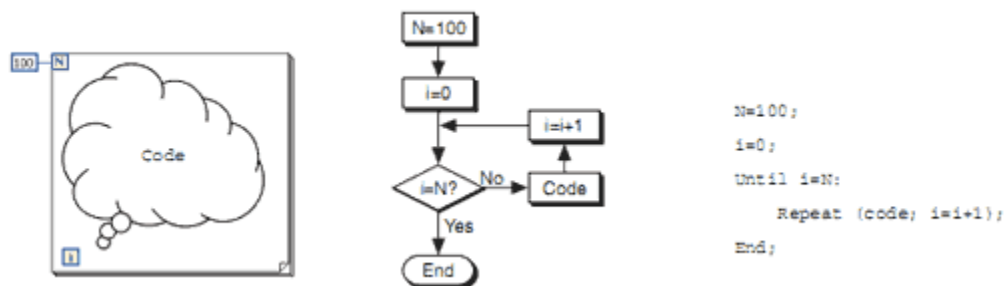


Figure 5. For Loop in LabVIEW, a Flowchart Equivalent of the For Loop Functionality, and a Pseudo Code Example of the Functionality of the For Loop.

The For Loop is located on the **Structures** palette. You also can place a While Loop on the block diagram, right-click the border of the While Loop, and select **Replace with For Loop** from the shortcut menu to change a While Loop to a For Loop.

N

The **Loop Count** terminal is an input terminal whose value indicates how many times to repeat the subdiagram.

i

The **Iteration Terminal** is an output terminal that contains the number of completed iterations. The iteration count for the For Loop always starts at zero.

The For Loop differs from the While Loop in that the For Loop executes a set number of times. A While Loop stops executing only if the value at the **Conditional Terminal** exists.

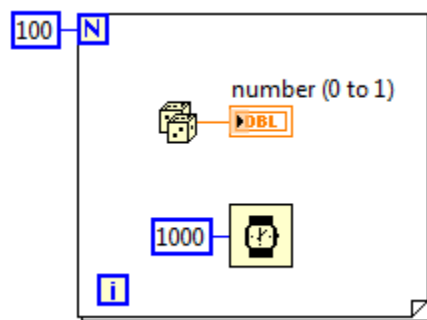


Figure 6. For Loop Example

Adding Timing to Loops

When a loop finishes executing an iteration, it immediately begins executing the next iteration, unless it reaches a stop condition. Most often, you need to control the iteration frequency or timing. For

example, if you are acquiring data, and you want to acquire the data once every 10 seconds, you need a way to time the loop iterations so they occur once every 10 seconds.

Wait Function

Place a wait function inside a loop to allow a VI to sleep for a set amount of time. This allows your processor to address other tasks during the wait time. Wait functions use the millisecond clock of the operating system.

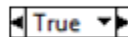


Case Structures

A Case structure has two or more subdiagrams, or cases. Only one subdiagram is visible at a time, and the structure executes only one case at a time.



An input value determines which subdiagram executes. The Case structure is similar to switch statements or if...then...else statements in text-based programming languages.



The **Case Selector** label at the top of the Case structure contains the name of the selector value that corresponds to the case



Wire an input value, or selector, to the **Selector Terminal** to determine which case executes. You must wire an integer, Boolean value, string, or enumerated type value to the **Selector Terminal**. You can position the **Selector Terminal** anywhere on the left border of the Case structure. If the data type of the selector terminal is Boolean, the structure has a TRUE case and a FALSE case. If the **Selector Terminal** is an integer, string, or enumerated type value, the structure can have any number of cases.

Note: By default, string values you wire to the selector terminal are case sensitive. To allow case-insensitive matches, wire a string value to the selector terminal, right-click the border of the Case structure, and select Case Insensitive Match from the shortcut menu.

You cannot specify a default case if you wire a Boolean control to the selector. If you right-click the case selector label, **Make This The Default Case** does not appear in the shortcut menu. Make the Boolean control TRUE or FALSE to determine which case to execute.

Right-click the Case structure border to **add, duplicate, remove, or rearrange cases** as well as select a **default case**.

Input and Output Tunnels

You can create multiple input and output tunnels for a Case structure. Inputs are available to all cases, but cases do not need to use each input. However, you must define an output tunnel for each case.

Consider the following example: a Case structure on the block diagram has an output tunnel, but in at least one of the cases, there is no output value wired to the tunnel. If you run this case, LabVIEW does not know which value to return for the output. LabVIEW indicates this error by leaving the center of the tunnel white. The unwired case might not be the case that is visible on the block diagram.

To correct this error, display the case(s) that contain(s) the unwired output tunnel and wire an output to the tunnel. You also can right-click the output tunnel and select **Use Default If Unwired** from the shortcut menu to use the default value for the tunnel data type for all unwired tunnels. When the output is wired in all cases, the output tunnel is a solid color.

Avoid using the **Use Default If Unwired** option. Using this option does not document the block diagram well, and can confuse other programmers using your code. The **Use Default If Unwired** option also makes debugging code difficult. If you use this option, be aware that the default value used is the default value for the data type that is wired to the tunnel. For example, if the tunnel is a Boolean data type, the default value is FALSE.

Data Type	Default Values
Numeric 	0 
Boolean 	FALSE 
String 	empty ("") 

Table 1. Data Type Default Values

Other Structures

LabVIEW has other, more advanced types of execution structures such as Event structures (used to handle interrupt-driven tasks like UI interaction) and Sequence structures (used to force execution order).

Passing Data between Loop Iterations in LabVIEW

When programming with loops, you often must access data from previous iterations of the loop in LabVIEW. For example, if you are acquiring one piece of data in every iteration of a loop and must average every five pieces of data, you must retain the data from previous iterations of the loop.

Shift Registers

Use shift registers when you want to pass values from previous iterations through the loop to the next iteration. A shift register appears as a pair of terminals directly opposite each other on the vertical sides of the loop border.



The terminal on the right side of the loop contains an up arrow and stores data on the completion of an iteration. LabVIEW transfers the data connected to the right side of the register to the next iteration. After the loop executes, the terminal on the right side of the loop returns the last value stored in the shift register.

Create a shift register by right-clicking the left or right border of a loop and selecting **Add Shift Register** from the shortcut menu.

A shift register transfers any data type and automatically changes to the data type of the first object wired to the shift register. The data you wire to the terminals of each shift register must be the same type.

You can add more than one shift register to a loop. If you have multiple operations that use previous iteration values within your loop, use multiple shift registers to store the data values from those different processes in the structure, as shown in Figure 8.

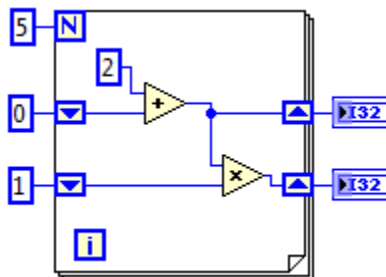


Figure 8. Using Multiple Shift Registers

Initializing Shift Registers

Initializing a shift register resets the value the shift register passes to the first iteration of the loop when the VI runs. Initialize a shift register by wiring a control or constant to the shift register terminal on the left side of the loop, as shown in Figure 9.

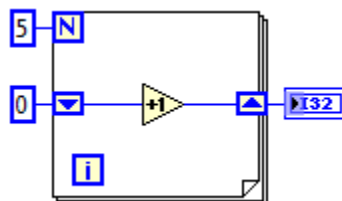


Figure 9. Initialized Shift Register

In Figure 9, the For Loop executes five times, incrementing the value the shift register carries by one each time. After five iterations of the For Loop, the shift register passes the final value, 5, to the

indicator and the VI quits. Each time you run the VI, the shift register begins with a value of 0. If you do not initialize the shift register, the loop uses the value written to the shift register when the loop last executed or, if the loop has never executed, the default value for the data type. Use an uninitialized shift register to preserve state information between subsequent executions of a VI.

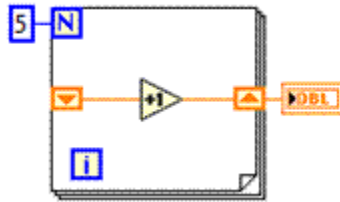


Figure 10. Uninitialized Shift Register

In Figure 10, the For Loop executes five times, incrementing the value the shift register carries by one each time. The first time you run the VI, the shift register begins with a value of 0, which is the default value for a 32-bit integer. After five iterations of the For Loop, the shift register passes the final value, 5, to the indicator, and the VI quits. The next time you run the VI, the shift register begins with a value of 5, which was the last value from the previous execution. After five iterations of the For Loop, the shift register passes the final value, 10, to the indicator. If you run the VI again, the shift register begins with a value of 10, and so on. Uninitialized shift registers retain the value of the previous iteration until you close the VI.

Stacked Shift Registers

With stacked shift registers, you can access data from previous loop iterations. The registers remember values from multiple previous iterations and carry those values to the next iterations. To create a stacked shift register, right-click the left shift register terminal and select **Add Element** from the shortcut menu.

Stacked shift registers can occur only on the left side of the loop because the right terminal transfers the data generated from only the current iteration to the next iteration.

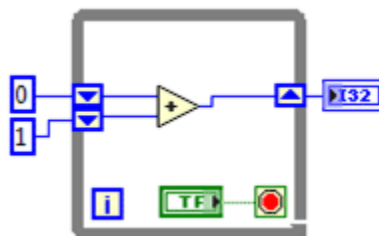


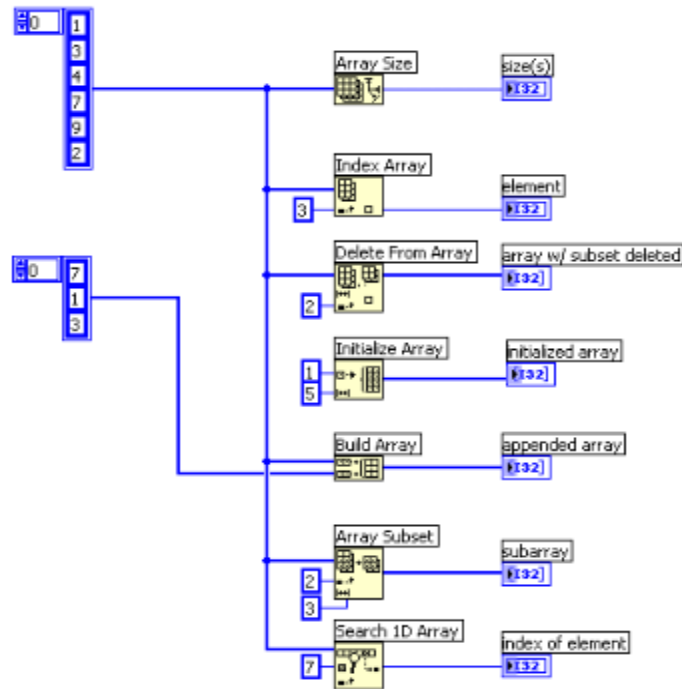
Figure 11. Using Stacked Shift Registers

If you add another element to the left terminal in the previous block diagram, values from the last two iterations carry over to the next iteration, with the most recent iteration value stored in the top shift register. The bottom terminal stores the data passed to it from the previous iteration.

Lab Tasks

In-Lab Task 1:

Implement the following VI with array control as input instead of array constant.

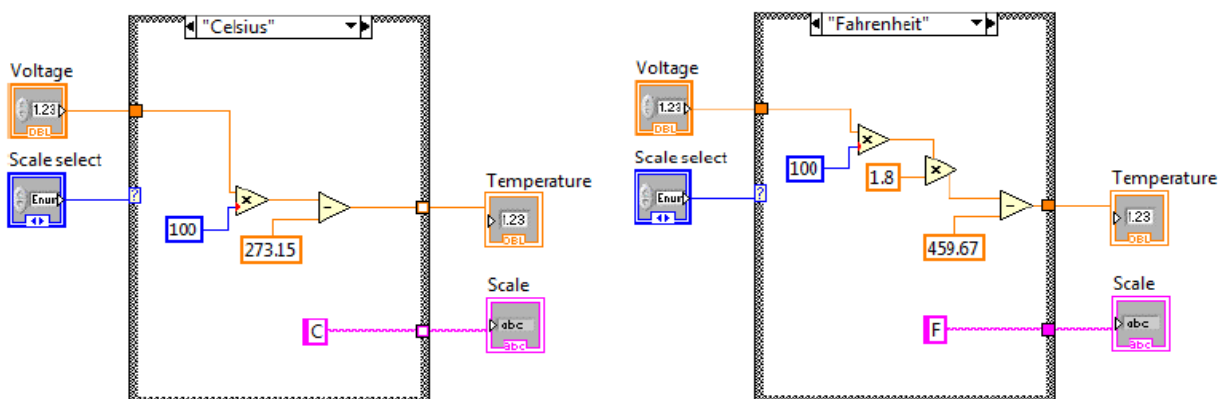


In-Lab Task 2:

Plot sine wave having 10Hz frequency both in LabVIEW Mathscript and Mathscript node.

In-Lab Task 3:

Use the case structure and enum to implement the following temperature conversion:



Note:

- Input should be converted into Celsius, and Fahrenheit.
- Input voltage is multiplied to 100 to amplify it.

Post-Lab Task 1:

Use the case structure to implement the following:

If voltage generated is greater than 15v then over limit Boolean indicator should get on.

Note:

- Voltage should be generated by random number function.
- When voltage gets low below 15v then over limit indicator should get off.

Post-Lab Task 2:

Create a subVI that converts kelvin temperature to Celsius and Fahrenheit.

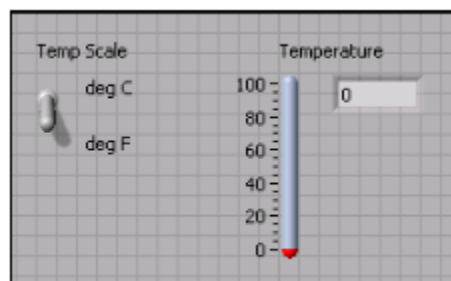


Figure 1. Front Panel

Note:

- Use this subVI in another VI.
- VI should have one input and output.
- Use case structure for different cases.
- Front panel of SubVI should look like as shown in figure 1.
- Create a suitable icon editor as shown below in figure 2.

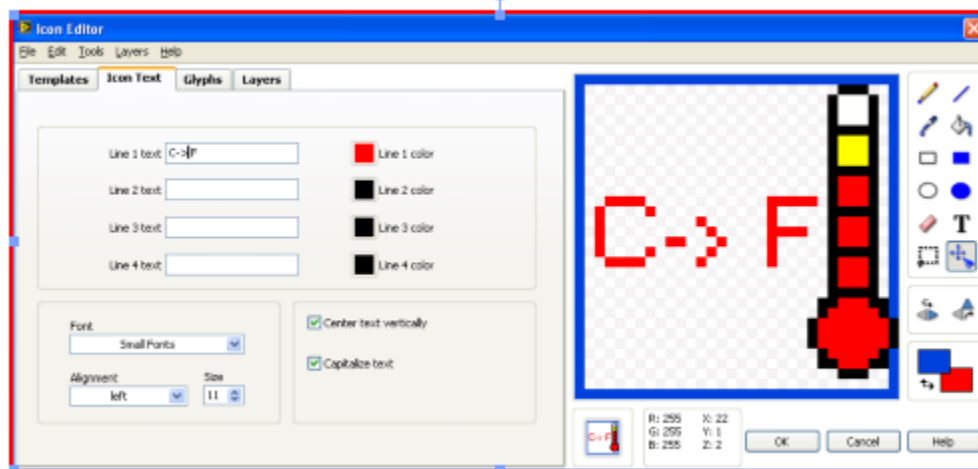


Figure 2. SubVI Icon

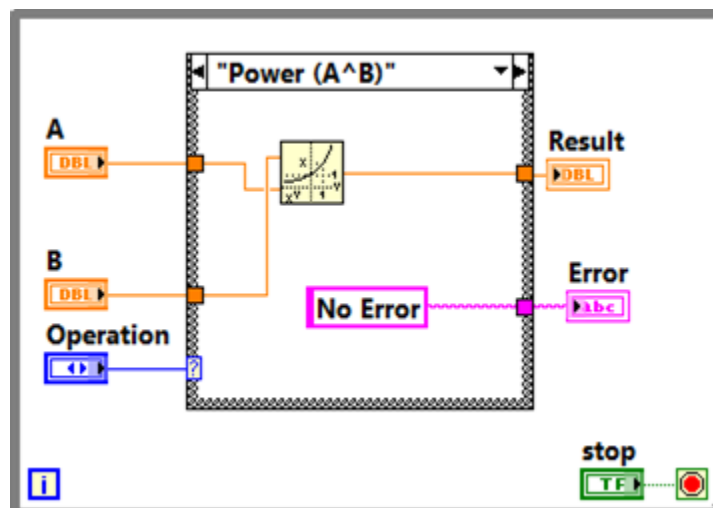
Useful formulae:

- $C = K - 273$
- $F = 1.8(K - 273) + 32$

Post-Lab Task 3:

Design a calculator in LabVIEW that will add, subtract, divide, multiply, divide, take the square root, and raise to a power. Add the following features also

- Add functionality for trigonometric functions such as sine, cosine, and tangent.
- Add a conversion from angle in degrees to radians.
- Add a delay to the While Loop to decrease the CPU usage.



Note:

- Should this program run one time or continuously? Do you want to press the Run button every time you need to calculate something?
- You should have only two inputs to the calculator.
- You should perform only one operation at a time. What could you use to contain all of the different operations but run only one at a time?
- Can you think of a control in which you can store multiple text strings that the user can select during run time?
- Only take the square root of the first input.
- Only use first input to convert it to sine, cosine and tangent.
- First input is used to convert it to radian from degree.
- Should the two inputs allow for negative numbers? What if a user inputs a negative number for the square root? Can you think of a way to check if the number is negative and how to account for this in your program?
- Should you indicate an error if the user tries to take the square root of a negative number?
- Search for *power* to locate a function to raise X to the power of Y.